

Model selection and evaluation

Quick recall:

Overfitting: when a model memorizes training data but fails on new data

- Small k in k -NN = **overfitting** (too sensitive to noise)
- Large k = **underfitting** (too smooth, misses patterns)

Challenge:

How do we systematically find the right k ?

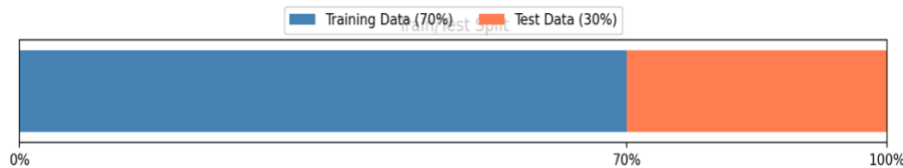
Key Question:

How do we actually measure whether our model generalizes?

Train/Test Split

We can't evaluate on data we trained on — of course it does well on that!

Solution: Hold out some data for testing.



- *Train* the model on the **training data** only
- *Evaluate* on the held-out **test data**
- **Test accuracy** tells us how well the model generalizes

$K=1$	100%	86%
$K=2$	95%	92%
$K=3$	94%	91%

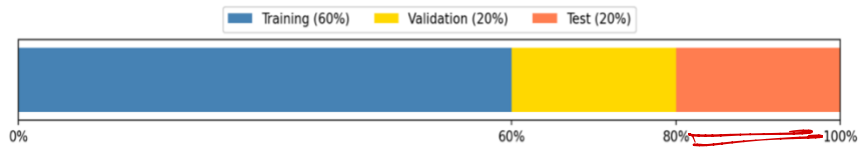
70% 69%

92% 89%

Train/Validation/Test Split

What if we tune k using the test set? We might overfit to the test set!

Solution: Three-way split



- **Train:** fit the model
- **Validation:** tune hyperparameters (like k)
- **Test:** final evaluation (touch only once!)

Why keep test data separate?

If you repeatedly check test accuracy while tuning, you're indirectly fitting to the test set.

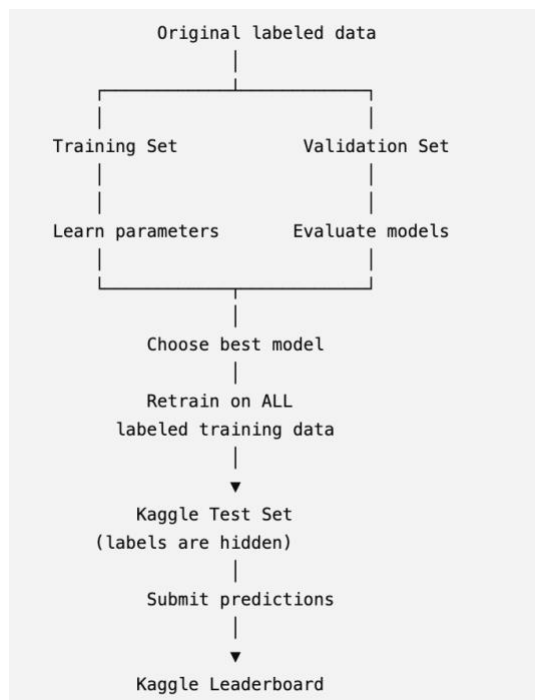
Example: Train/Validate/Test

Kaggle competitions illustrate this perfectly:

- **Public leaderboard:** you can submit and see your score (like validation)
- **Private leaderboard:** revealed only at the end (the true test)

Suppose Kaggle gives you **10,000 labeled houses**. If you train on all 10,000, how do you know whether your model is actually good?

You don't. Instead, you pretend that some of the answers are unknown. (say 8000-2000 split!)



Why retrain on all the labeled data?

After you've used the validation set to choose the best model, you no longer need to keep those examples separate. Since their labels are known, you can retrain the chosen model on **all 10,000 labeled houses** before making predictions on Kaggle's test set. This gives the model more data to learn from and often improves performance.

Example: Train/Validate/Test

Think of a Kaggle competition like preparing for a final exam:

- **Training set:** Homework and practice problems. You learn from these because the answers are provided.
- **Validation set:** Practice exam. You know the answers afterward, so you can identify mistakes and improve your study strategy.
- **Test set (Kaggle):** The actual final exam. You never see the answers—you only receive your score.

Think in terms of:

- **Training** is for **learning**.
- **Validation** is for **choosing and tuning** the model.
- **Test** is for **final evaluation** on completely unseen data.

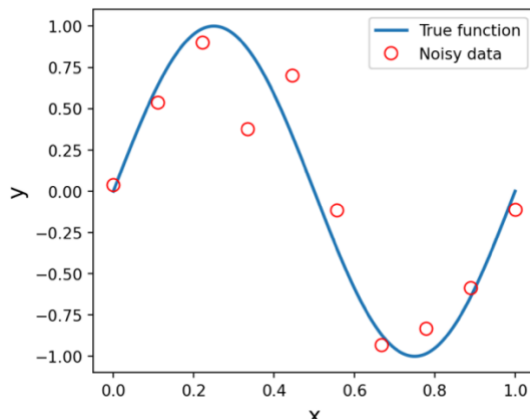
- You have 100 samples. What's wrong with using 95% for training and 5% for testing?
- A colleague says "I tried 20 different values of k and picked the one with the best test accuracy." What's the problem?

Understanding overfitting: Polynomial Regression example

Let's use polynomial regression to build intuition about overfitting.

This example makes overfitting very visible — then we'll apply the same ideas to k-NN.

Consider synthetic data generated from $y = \sin(2\pi x)$ plus noise:



We'll try to fit polynomials of different degrees:

$$y(x, \mathbf{w}) = w_0 + w_1x^1 + w_2x^2 + \cdots + w_kx^k$$

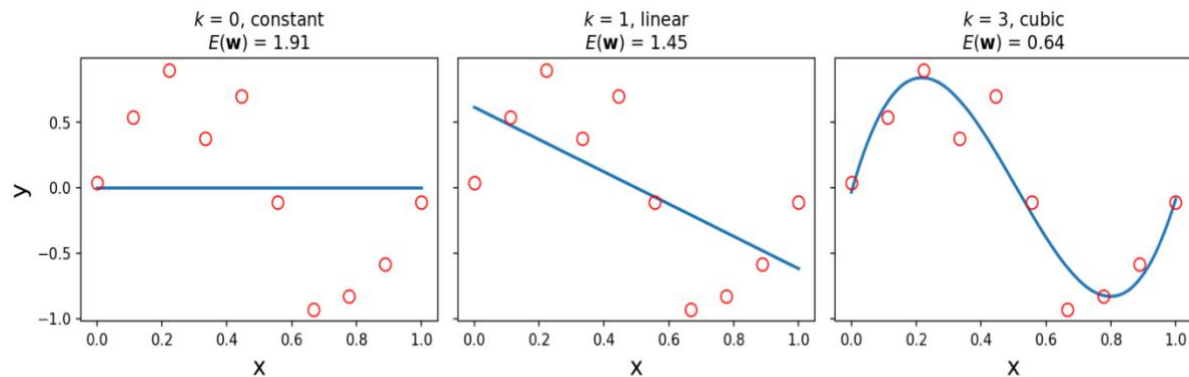
Here:

- k is the hyperparameter (we choose it)
- The w_i values are parameters (learned from the data)

Understanding overfitting: Polynomial Regression example

Model selection: Choosing k

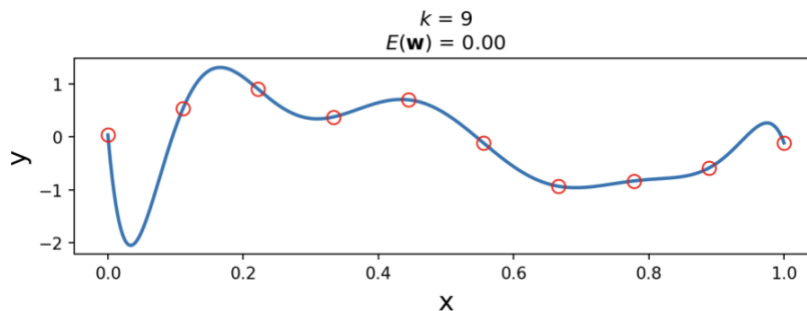
The problem of choosing the hyperparameter k is called model selection. Let's apply regression to build the best possible constant (order 0), linear (order 1), and cubic (order 3) models.



Here $E(\mathbf{w})$ is the **training error**.

A third-order polynomial ($k = 3$) leads to small training error and hence is a good fit. What about higher order polynomials?

By setting $k = 9$, we get the following polynomial to fit the data:



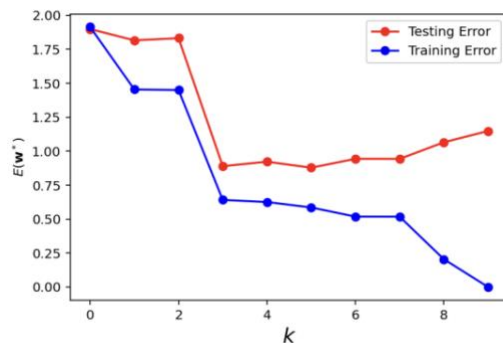
Is the 9th order polynomial a "better" model for this dataset?

Understanding overfitting: Polynomial Regression example

Model selection: Choosing k

We want the model to have a good generalization ability (i.e., do a good job of predicting on future data).

The 9th degree polynomial has poor generalization ability for this dataset.



Test data is data that was not used to train the model but comes from the same source.

As polynomial order increases:

- **Training error** always *decreases* (eventually hits zero)
- **Test error** *decreases*, then *increases* again

The best test error is at $k = 3$ (cubic). Higher-order polynomials **overfit**

This is the core tradeoff: **fitting** training data well vs. **generalizing** to new data.

Two ways to look at it:

- Too many parameters for the amount of data: We can see this in the fact that we have only 10 training points, and the 9th order polynomial has 10 coefficients.
- Model too complex for the underlying pattern — the model is not just fitting the underlying phenomenon, but also the noise in the data.

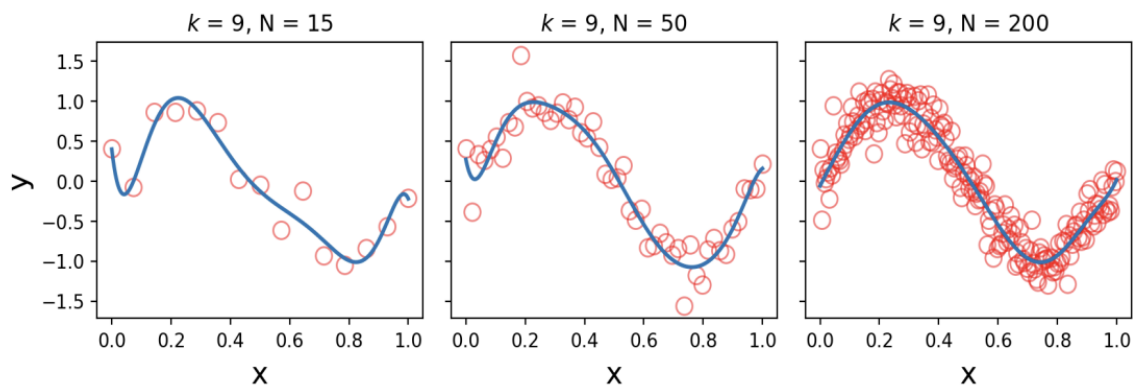
These suggest techniques we may use to avoid overfitting:

- Increase the amount of training data. All else being equal, more training data is always better
- Limit the complexity of the model. Model complexity is often controlled via hyperparameters.

More Training Data

It's not necessarily true that an order-3 polynomial is best for this problem. But the higher the order of polynomial we want to fit, the more data we need to avoid overfitting.

Here we use an order-9 polynomial for increasing amounts of training data ($N = 15, 50, 200$):



Hyperparameters

Getting more data helps but we can't always get more data!

Instead: we *control model complexity*!

A hyperparameter is a setting we choose before training:

- Polynomial degree k
- Number of neighbors k in k -NN
- Learning rate, regularization strength, ...

How do we choose the right value? *Hold out validation data and compare!*

But

A single random split can be lucky or unlucky!

Solution 1: Random Subsampling — do multiple random splits and average the results.

Solution 2: Cross-Validation — systematically use all data for both training and testing.

Random Subsampling

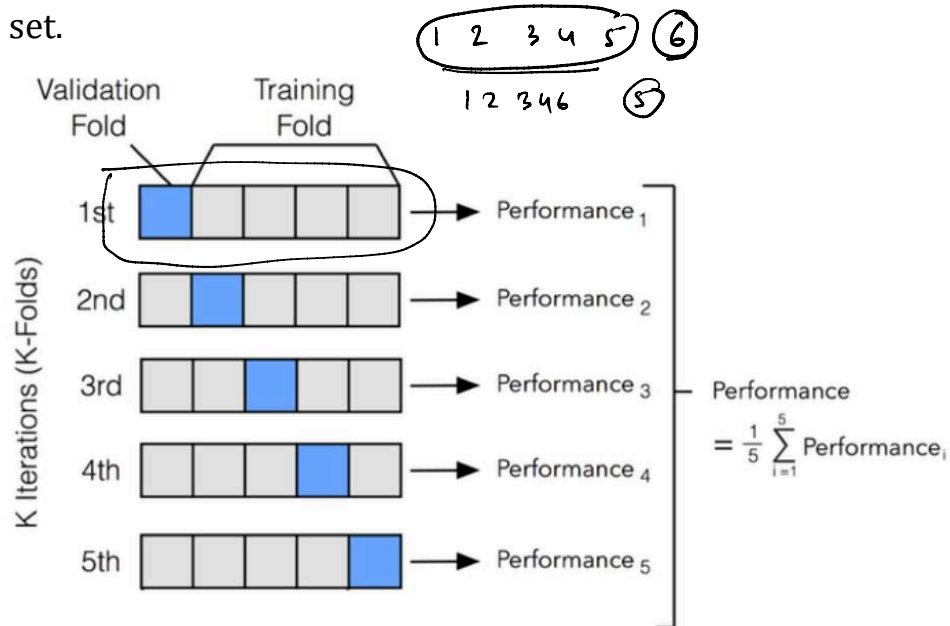
For each hyperparameter value:

- Randomly split data 5+ times
- Train and evaluate on each split
- Average the results

More robust than a single split — but there's an even better approach...

Cross Validation

k-fold cross-validation: split data into parts. Each part takes a turn as a validation set.



The value of k can vary up to the size of the dataset.

The **larger** k we use, the more data is used for training, but the more folds must be evaluated, which **increases the time** required.

In the extreme case where **k is equal to the data size**, then each data item is held out by itself; this is called "**leave-one-out**".

Common defaults: 5-fold, 10-fold: good balance!

You're doing 5-fold cross-validation on a dataset of 1000 samples (assume you've already set aside a test set of 200, and you have 1000 left).

1. How many samples are in each fold?
2. How many times do you train the model?
3. How many samples does each model see during training?

1) 5-fold $\therefore \frac{1000}{5} = 200$ samples in each fold

2) 5 times

3) During training, the model trains on 4 folds

$$\therefore 4 \times 200 = 800 \text{ samples}$$

Grid search with cross validation

Suppose we use 5-fold cross validation. So, our data is divided into 5 equal pieces.

Eventually every fold will become the validation set once.

Suppose for $k = 3$

Fold $\rightarrow$ iterations	Accuracy %
1	94
2	96
3	95
4	93
5	97

Average accuracy: 95%

Now, suppose for $k = 5$

Fold	Accuracy %
1	96
2	98
3	97
4	95
5	96

Average accuracy: 96.4%

Repeat for every candidate value of k .

Grid: Simply means all combinations of hyper parameters you want to try.

For our case, it may be all our candidate values of k : 1,3,5,7,9

Grid search + cross validation:

For every hyperparameter combination $\rightarrow$ perform 5-fold cross validation $\rightarrow$

Compute average validation accuracy $\rightarrow$ Keep the best one

Evaluating a binary classifier

Suppose we run the k -nearest neighbors algorithm on some dataset. How do we measure whether it has performed well or poorly?

That is: how do we evaluate a classifier?

In the simple case of a *binary classifier*-- meaning that there are exactly two possible labels -- we can call one class the '*Positive*' class and one the '*Negative*' class.

The most basic measure of success for a classifier is *accuracy*: what fraction of test points are correctly classified?

Example:

Dataset is 90% positive, 10% negative.

A model that always predicts “positive” gets 90% accuracy!

Is that a good model? *No*. We need better metrics.

Confusion Matrix

		Classifier Prediction	
		Positive	Negative
Actual Value	Positive	True Positive	False Negative
	Negative	False Positive	True Negative

Diagonal = correct, *Off-diagonal* = errors

Key metrics:

- **Recall** = $TP / (TP + FN)$: "Of all actual positives, how many did we catch?"
- **Precision** = $TP / (TP + FP)$: "Of all predicted positives, how many were right?"

Recall ↑ False Negatives ↓
Precision ↑ False Positives ↓

Precision vs Recall: A tradeoff

Suppose, you are building a cancer screening test:

	Predicted +	Predicted -
Actual +	80	20
Actual -	10	90

Precision = $80/90 \approx 89\%$. Recall = $80/100 = 80\%$.

Which matters more here: fewer false alarms, or fewer missed cancers?

Precision: Out of the predicted, how many were right?
More precision: Fewer false positives: Fewer false alarms

Recall: Out of the actual cases, how many were caught?

More Recall: Fewer false negatives: Less Missed Cancers
↓
More preferable

Can think of a scenario where precision would be favored?



More metrics

From the confusion matrix:

	Predicted +	Predicted -
Actual +	TP	FN
Actual -	FP	TN

Key rates:

- **True Positive Rate (TPR)** = Recall = Sensitivity

$$TPR = \frac{TP}{TP + FN}$$

- **False Positive Rate (FPR)**

$$FPR = \frac{FP}{FP + TN}$$

- **Specificity** = 1 - FPR

$$Specificity = \frac{TN}{FP + TN}$$



The F1 Score

What if you want a single number that balances precision and recall?

F1 Score: The *harmonic mean* of precision and recall

$$F1 = 2 \cdot \frac{Precision \cdot Recall}{Precision + Recall} = \frac{2 \cdot TP}{2 \cdot TP + FP + FN}$$

Why harmonic mean?

It penalizes extreme imbalance.

Precision	Recall	Arithmetic Mean	F1 Score
0.90	0.90	0.90	0.90
0.99	0.01	0.50	0.02

A model that ignores one metric can't hide behind a high average!

When to use which metric

Scenario	Prioritize	Why?
Cancer Screening	Recall	Don't miss actual cases
Spam Filter	Precision	Don't block legitimate emails
Balanced Importance	F1 Score	Single balanced metric
Understanding errors	Confusion Matrix	See full error breakdown

Your turn!

A model for detecting fraudulent transactions gives these results:

	Predicted: Fraud	Predicted: Legit
Actual: Fraud	45	5
Actual: Legit	30	920

Calculate:

1. Accuracy
2. Precision (for detecting fraud)
3. Recall (for detecting fraud)

Which metric matters most for fraud detection? Why?

1. Accuracy: What %age of our predictions were

correct:
$$\frac{TP + TN}{TP + TN + FP + FN} = \frac{45 + 920}{45 + 920 + 30 + 5}$$

$$= \frac{965}{1000} = 96.5\%$$

Precision for fraud detection:
Out of all predicted as fraud, how many were actually fraud? :
$$\frac{TP}{TP + FP} = \frac{45}{45 + 30} = 60\%$$

Recall for fraud detection:
Out of all actually fraud, how many were caught? :
$$\frac{TP}{TP + FN} = \frac{45}{45 + 5} = 90\%$$

Because we are trying to catch fraud transactions, having fewer false negatives is more desirable.
∴ Recall is more important